

Git Assignment 1

Collaborative Product Development

A complete GitLab workflow for a distributed food delivery startup, from idea to production release

Field	Details
Student name	Mohit
Institution	National Institute of Information Technology
Course	Java Full stack
Instructor	Mr. Praveen
Submission date	October 03, 2026

Table of Contents

1. Executive Summary	3
2. Business Scenario and Objectives.....	3
2.1 Scenario.....	3
2.2 Objectives.....	3
3. Stakeholders.....	3
4. Current Challenges and Risks	4
5. Assumptions and Constraints	4
5.1 Given Requirements (from the assignment)	4
5.2 Assumptions (made by the author)	5
5.3 Constraints.....	5
6. Proposed Solution (Step-by-Step Approach).....	5
6.1 Set Up the Group and Projects	6
6.2 Capture Ideas as Issues, Labels and Milestones.....	6
6.3 Plan the Work on an Issue Board.....	7
6.4 Choose a Branching Strategy.....	7
6.5 Merge Requests and Code Review.....	8
6.6 Enforce Governance with Protected Branches	10
6.7 Test and Build Automatically (CI/CD).....	10
6.8 Release and Tag to Production.....	11
7. Presentation and Justification	11
Why this approach is justified	12
8. Implementation Plan.....	12
Training and communication	13
9. Measurable Success Criteria	13
10. Recommendations and Lessons Learned	13
10.1 Recommendations.....	13
10.2 Lessons Learned	13

1. Executive Summary

Problem. A startup is building a food delivery application with developers spread across different cities. Without one shared way of working, people duplicate effort, merge risky code, lose track of issues and release unfinished features.

Solution. One GitLab workflow covers the whole journey from idea to production. Ideas become **issues**, issues are sorted with **labels** and grouped into **milestones**, and work is planned on an **issue board**. Code is written on short-lived **feature branches**, merged only through **merge requests (MRs)** with **code review**, protected by **protected branches**, **approval rules** and **CI/CD pipelines**, and shipped through **release tags** and **GitLab Releases**.

Why it works. Every change is traceable from idea to release, nothing reaches production without review and a passing pipeline, and teams in different cities see the same board and follow the same rules.

Note on terms. GitLab calls a pull request a **merge request**, and a GitHub organization a **group**. This report uses the GitLab names.

2. Business Scenario and Objectives

2.1 Scenario

A startup is building a food delivery application (customer app, restaurant dashboard, delivery partner app and backend services). Development is distributed across different cities, so teams cannot rely on talking at the same desk. They need issue tracking, merge requests, code reviews, project boards, labels, milestones, branch protection and release tagging, and a workflow that enforces governance and collaboration standards.

2.2 Objectives

#	Objective	Why it matters
O1	Track every piece of work as an issue	Nothing is lost or done twice across cities
O2	Merge code only through reviewed merge requests	Catches bugs and shares knowledge
O3	Protect main and release branches	Production code stays stable
O4	Give everyone one view of progress (boards, milestones)	Remote teams stay aligned
O5	Release in a controlled, versioned way	Easy to trace and roll back
O6	Enforce governance with rules, not reminders	Standards hold even as the team grows

3. Stakeholders

A stakeholder is anyone affected by, or responsible for, the outcome of the workflow.

Stakeholder	Role	What they need
Product owner	Decides features and priorities	Clear backlog and visible progress
Engineering manager	Leads the distributed teams	Predictable delivery and fair workload
Developers (all cities)	Build features and fix bugs	Clear tasks and fast reviews

Stakeholder	Role	What they need
Reviewers / maintainers	Review and approve merge requests	Small, well-described merge requests
QA / Testers	Verify quality before release	Stable builds and a list of what changed
DevOps engineer	Runs CI/CD and deployments	Reliable pipelines and tagged releases
Designers	Design screens and flows	Issues that link to designs
Security lead	Protects customer and payment data	Reviews, secret detection, audit trail
Restaurants and delivery partners	Use the platform daily	Stable app with few outages
Customers	Order food	A fast, correct, reliable app

4. Current Challenges and Risks

#	Challenge	Impact	Risk
C1	Work is not tracked in one place	Duplicate or forgotten tasks	High
C2	Direct pushes to main	Broken code reaches production	High
C3	Code is merged without review	Bugs and security holes slip through	High
C4	Teams in different time zones	Slow reviews and blocked work	Medium
C5	Long-lived branches	Large, painful merge conflicts	Medium
C6	No consistent labels or priorities	Hard to filter, plan or report	Medium
C7	Unversioned releases	Cannot tell what is live or roll back	High
C8	Secrets (API keys, payment keys) committed to code	Data breach and financial loss	High
C9	Unclear who owns which code	Wrong or no reviewer	Medium

Risk level is a judgement based on the likely effect on a food delivery business (orders, payments, reputation). It is not measured data.

5. Assumptions and Constraints

This section separates what the assignment **requires** from what this report **assumes**.

5.1 Given Requirements (from the assignment)

#	Requirement
R1	A startup is building a food delivery application
R2	Development is distributed across different cities
R3	Teams need issue tracking, merge (pull) requests, code reviews, project boards, labels, milestones, branch protection and release tagging
R4	Design a complete workflow from idea to production release
R5	Enforce governance and collaboration standards
R6	Identify stakeholders and objectives; analyse challenges and risks
R7	Create a step-by-step solution approach; document assumptions and constraints
R8	Prepare presentation and justification; define measurable success criteria

#	Requirement
R9	Deliver a detailed report, workflow diagrams, recommendations, lessons learned and an implementation plan

5.2 Assumptions (made by the author)

#	Assumption	Effect if wrong
A1	Project, group and team names in this report are illustrative examples	Real names differ; the method stays the same
A2	GitLab is the platform, on a Premium plan or higher (approval rules, code owner approvals, push rules and scoped labels need it)	On the Free plan some rules become team agreements instead of enforced settings
A3	There are about 20 to 30 developers in 3 to 4 cities, plus QA and DevOps	Review rules and team sizes would change
A4	GitLab CI/CD (.gitlab-ci.yml) with shared runners is used for pipelines	Another CI tool would replace the pipeline file
A5	The app is split into a few projects (mobile, web, backend)	A single monorepo would need path-based CODEOWNERS
A6	Teams have at least 2 to 3 hours of overlap per day	Review time targets would be longer
A7	Targets in Section 9 are illustrative because no baseline data is given	Re-base targets on real data

5.3 Constraints

- **Customer and payment data:** code must never expose secrets or personal data.
- **Time zones:** reviews and handovers must work without live meetings.
- **Startup speed:** rules must protect quality without slowing delivery too much.
- **Plan limits:** some GitLab features depend on the subscription tier chosen.
- **Scope:** this is a design and planning exercise; no real systems are changed.

6. Proposed Solution (Step-by-Step Approach)

The solution has multiple steps, taken in order. Together they take work from an idea to a tagged production release.

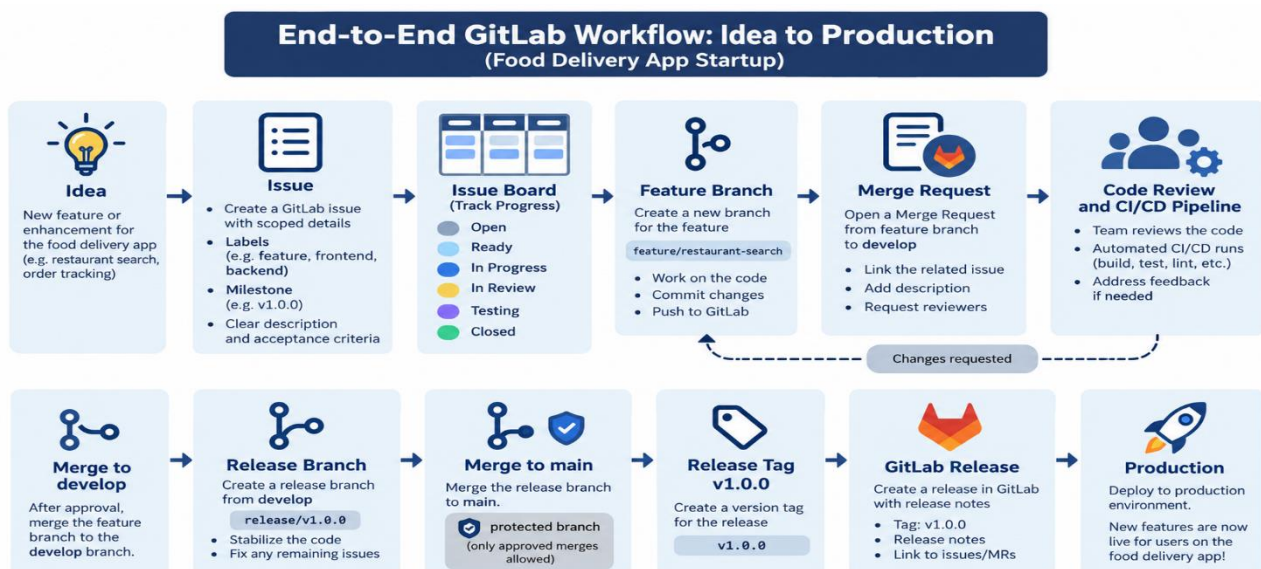


Diagram 1: END-TO-END GITLAB WORKFLOW (IDEA TO PRODUCTION)

6.1 Set Up the Group and Projects

Simple explanation: a GitLab **group** is the company space. Inside it, **subgroups** organise teams and **projects** hold the code. Members get a **role** that decides what they can do.

Item	Example (illustrative)	Purpose
Group	quickbite-foods	One home for all code and people
Subgroups	mobile, backend, platform, qa	Organise teams and share settings
Projects	customer-app, restaurant-dashboard, delivery-app, backend-api, infra	One project per deployable part
Roles	Guest, Reporter, Developer, Maintainer, Owner	Least privilege: people get only what they need
Files in every project	README.md, CONTRIBUTING.md, CODEOWNERS, issue and MR templates (.gitlab folder), .gitlab-ci.yml	Same standards everywhere

6.2 Capture Ideas as Issues, Labels and Milestones

Simple explanation: an **issue** is one task, bug or idea. **Labels** sort issues. A **milestone** groups issues that must be done for a release or sprint. Labels and milestones are created once at **group level** so every project shares them.

Label group	Scoped labels (example)	Use
Type	type::feature, type::bug, type::chore, type::docs	What kind of work
Priority	priority::critical, priority::high, priority::medium, priority::low	What to do first
Area	area::ordering, area::payments, area::tracking, area::restaurant	Which part of the app
Status	status::ready, status::in-progress, status::in-review, status::testing	Where it is on the board
Help	good-first-issue, help-wanted	Onboarding new developers

Scoped labels use a double colon, and an issue can have only one label from each scope (for example, one priority). On plans without scoped labels, plain names such as type-bug work the same way.

Milestones follow releases, for example v1.0 MVP (menu, cart, checkout), v1.1 Live Tracking and v1.2 Offers and Coupons. Each has a due date and GitLab shows the percentage complete. Big features can be grouped in an **epic**.

Issue template (saved in .gitlab/issue_templates/Feature.md). The last two lines are GitLab quick actions that apply labels and milestone automatically:

```
## Problem
Customers cannot see where their order is.

## Acceptance criteria
- [ ] Map shows the rider location, updated every 10 s
- [ ] Works offline gracefully
```

```
/label ~"type::feature" ~"area::tracking" ~"priority::high"
/milestone %"v1.1 Live Tracking"
```

6.3 Plan the Work on an Issue Board

Simple explanation: a GitLab **issue board** is a Kanban board. Each column is a label, so every city sees the same picture of the work.

Column	Meaning	Issue moves when
Open (Backlog)	Ideas and approved issues, not yet planned	Issue is created and labelled
Ready	Clear, sized and assigned for the sprint (status::ready)	Acceptance criteria are agreed
In progress	A developer is working on it	A branch and draft MR are created
In review	Merge request is ready for review	Draft status removed
Testing	Merged to develop and being tested by QA	MR merged to develop
Closed (Done)	Released or verified	Release tag published

An issue **closes automatically** when a merge request with "Closes #142" is merged into the project's **default branch**. For this reason develop is set as the default branch. Label changes use drag and drop, quick actions or a triage bot. Boards can be filtered by city, label, milestone or assignee.



Diagram 2: ISSUE BOARD LIFECYCLE

6.4 Choose a Branching Strategy

Simple explanation: a **branch** is a safe copy of the code to work on. The team uses a simple, well-known model with two long-lived branches and many short-lived ones.

Branch	Lifetime	Rule
main	Permanent	Always production-ready; only release merges; every release is tagged
develop	Permanent (default)	Integration branch; all feature work merges here first
feature/<issue>-<name>	Short (days)	Created from develop; one issue per branch
bugfix/<issue>-<name>	Short	Fixes found in develop or staging
release/<version>	Short	Final testing before a release
hotfix/<issue>-<name>	Very short	Urgent production fix from main; merged to main and develop

```
git checkout develop && git pull origin develop
git checkout -b feature/142-live-tracking
# work, commit often with clear messages
git commit -m "feat(tracking): show rider on map (#142)"
git push -u origin feature/142-live-tracking
```

GitLab also has a **Create merge request** button on an issue that makes the branch and a draft MR in one click. Commit messages follow a simple pattern (type, area, short summary, issue number). A push rule can require branch names to start with feature/, bugfix/, release/ or hotfix/.

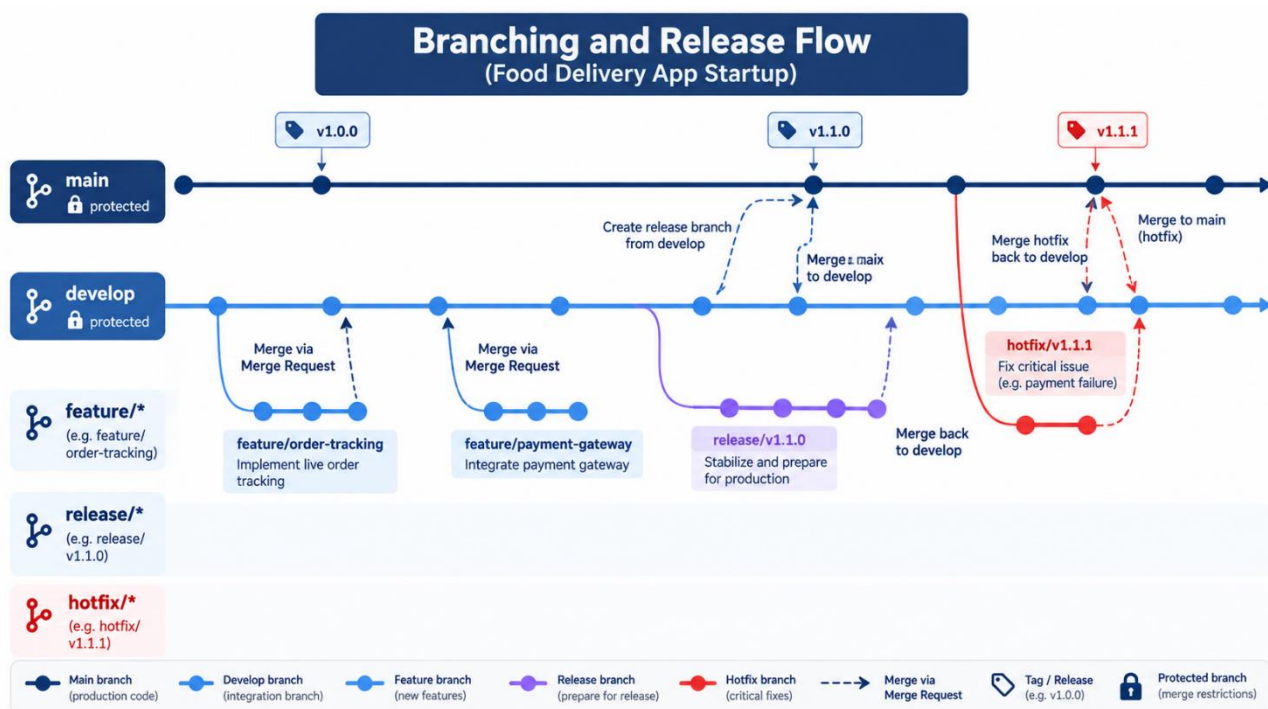


Diagram 3: BRANCHING AND RELEASE FLOW

6.5 Merge Requests and Code Review

Simple explanation: a **merge request (MR)** asks the team to review a branch before it is merged. Reviewers read the change, comment in **threads**, and approve or ask for changes.

1. **Open a draft MR** early from the feature branch into develop. Put "Closes #142" in the description so the issue closes automatically.
2. **Fill the MR template:** what changed, why, how it was tested, screenshots and a checklist.
3. **The pipeline runs automatically:** build, tests, lint and security scan, with results shown on the MR.

4. **Mark the MR ready.** Approvers are requested automatically from the CODEOWNERS file (for example, payments code needs the payments team).
5. **Review within 24 hours:** a reviewer in another city picks it up (follow-the-sun reviews). Reviewers use threads and **suggested changes**.
6. **Resolve all threads**, push fixes and collect the required approvals.
7. **Squash and merge** into develop. The source branch is deleted automatically.

Review rule	Detail
Size	Aim for MRs under about 400 changed lines; split larger work
Approvals	1 approval for normal changes; 2 for payments, security or release MRs
Author	Cannot approve their own MR
Review quality	Check logic, tests, security, naming and performance; be kind and specific
Stale approvals	New commits remove old approvals so the final code is reviewed

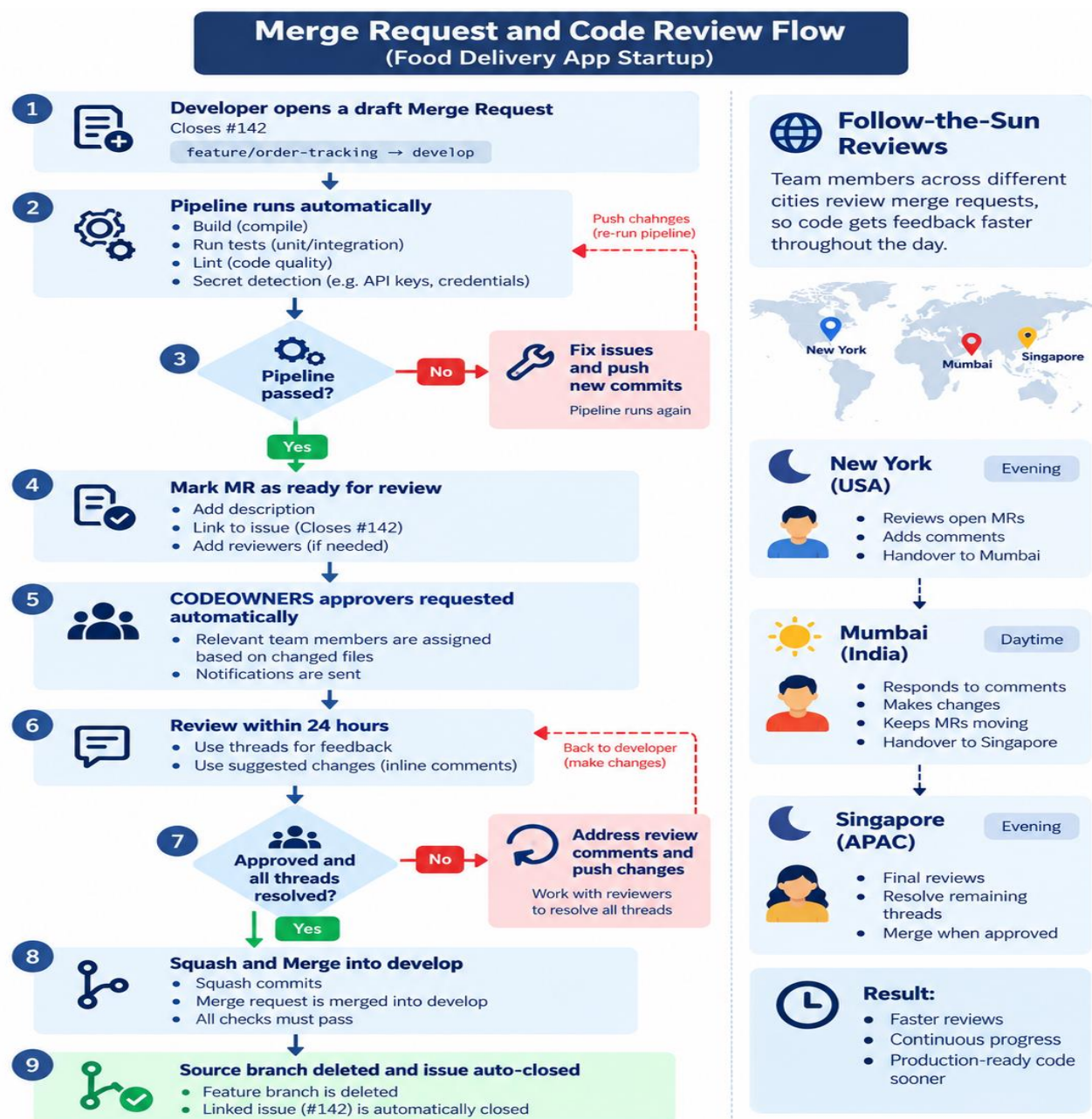


Diagram 4: MERGE REQUEST AND CODE REVIEW FLOW

6.6 Enforce Governance with Protected Branches

Simple explanation: **protected branches**, **approval rules** and **merge checks** turn the team's rules into settings that GitLab enforces, so they cannot be skipped by mistake.

Setting	main	develop
Allowed to push and merge directly	No one	No one
Allowed to merge (through an MR)	Maintainers	Developers and Maintainers
Allow force push	Off	Off
Required approvals	2	1
Code owner approval required	Yes	Yes
Prevent approval by the MR author	Yes	Yes
Remove approvals when new commits are pushed	Yes	Yes
Pipelines must succeed	Yes	Yes
All threads must be resolved	Yes	Yes
Squash commits	Off (keep release history)	Required
Reject unsigned commits (push rule)	Yes	Optional
Delete source branch after merge	Not applicable	Yes

Other governance settings:

- Two-factor authentication required for the whole group.
- **Protected tags:** only Maintainers can create tags that match v*.
- **Protected environments:** production deployment needs an approval.
- **Secret detection** in the pipeline (and push protection on plans that include it); API keys stored only as masked, protected CI/CD variables.
- **Audit events** record who changed settings, members and protections.

CODEOWNERS example (each section sets the default owners for the paths below it):

```
[Payments] @quickbite-foods/payments-team
/payments/

[Mobile] @quickbite-foods/mobile-team
/mobile/

[Infrastructure] @quickbite-foods/devops
/infra/
.gitlab-ci.yml
```

6.7 Test and Build Automatically (CI/CD)

A pipeline defined in **.gitlab-ci.yml** runs on every merge request, so reviewers see the results before reading the code.

Stage	What runs	Blocks merge?
Build	Compile the app (for example, Maven for the Java backend)	Yes
Test	Unit and integration tests	Yes
Lint	Style and static analysis	Yes
Security	Secret detection and dependency checks	Yes

Stage	What runs	Blocks merge?
Review app	Temporary preview environment for the MR	No
Deploy	Staging deploys automatically from develop; production deploys from a release tag after approval	Not applicable

```

stages: [build, test, security, deploy]

include:
  - template: Security/Secret-Detection.gitlab-ci.yml

build:
  stage: build
  script: ./mvnw -B package -DskipTests

test:
  stage: test
  script: ./mvnw -B verify

deploy-production:
  stage: deploy
  script: ./deploy.sh production
  environment: production
  rules:
    - if: $CI_COMMIT_TAG =~ /^v\d+\.\d+\.\d+$/
      when: manual

```

6.8 Release and Tag to Production

Simple explanation: a **tag** marks an exact commit as a version. A **GitLab Release** attaches release notes to that tag.

1. When the milestone is complete, create **release/1.1.0** from develop.
2. Run final QA on staging; only bug fixes are allowed on the release branch.
3. Open an MR from release/1.1.0 into **main** (2 approvals, pipeline green).
4. Merge, then **tag** the merge commit using semantic versioning (v1.1.0).
5. Publish a **GitLab Release** with release notes (from the Releases page or a pipeline job); the tag pipeline offers the production deploy.
6. Merge main back into develop so the branches stay in sync.
7. Close the milestone and check that issues are closed.

```

git checkout main && git pull
git tag -a v1.1.0 -m "Release 1.1.0: Live tracking"
git push origin v1.1.0

# Urgent production bug: hotfix from main
git checkout -b hotfix/190-payment-timeout main
# fix, MR to main, tag v1.1.1, merge back into develop

```

Versioning: MAJOR.MINOR.PATCH. A new feature raises MINOR, a bug fix raises PATCH and a breaking change raises MAJOR. If a release fails, the team redeploys the previous tag.

7. Presentation and Justification

The solution is presented in a short deck to the product owner and engineering leads. The outline below shows what each slide says and why.

#	Slide	Key message
1	Problem	Distributed teams, no shared process, risk of broken releases
2	Stakeholders and goals	Who is affected and what success means
3	The workflow (Diagram 1)	Idea to issue to board to branch to MR to release
4	Planning tools	Issues, labels, milestones and the issue board
5	Code quality gates	MR review, CODEOWNERS and pipeline checks
6	Governance	Protected branches and approval rules enforced by GitLab
7	Release process	Tags, GitLab Releases, hotfix path and rollback
8	Results and plan	Success metrics and a 6-week rollout

Why this approach is justified

Risk (from Section 4)	How the workflow reduces it
Untracked work (C1)	Every task is an issue on a shared board
Direct pushes and no review (C2, C3)	Protected branches force reviewed merge requests
Time-zone delays (C4)	Follow-the-sun reviews, review time target, written MR templates
Merge conflicts (C5)	Short-lived branches and small MRs
Messy planning (C6)	Group-level labels, milestones and board columns
Unversioned releases (C7)	Semantic version tags and GitLab Releases
Leaked secrets (C8)	Secret detection, masked variables and protected variables
Unclear ownership (C9)	CODEOWNERS requests the right approvers automatically

Alternatives considered: trunk-based development with feature flags is faster but needs mature testing; a single shared branch is simplest but unsafe. The develop/main model is a good middle ground for a growing distributed team.

8. Implementation Plan

A rollout of about **six weeks** is assumed.

Phase	When	Activities	Output
1. Prepare	Week 1	Create group, subgroups and projects; enforce 2FA; agree workflow	Group ready
2. Standards	Week 1 to 2	Add README, CONTRIBUTING, templates, CODEOWNERS and group labels	Standard project template
3. Protect	Week 2	Set protected branches, approval rules, merge checks, push rules and protected tags	Enforced rules
4. Automate	Week 3	Build .gitlab-ci.yml; connect runners and the staging deploy	Green pipeline
5. Plan	Week 3 to 4	Create milestones, import backlog as issues, set up the issue board	Live board
6. Pilot	Week 4 to 5	One team runs a full feature through the workflow; collect feedback	Pilot report
7. Roll out	Week 6	All teams adopt; first tagged release; short training session	v1.0.0 released

Phase	When	Activities	Output
8. Review	After 1 month	Retrospective and metric review	Improvements

Training and communication

- A 60-minute recorded walkthrough for all cities, repeated for each time zone if needed.
- A one-page cheat sheet with branch names, commit format and MR steps.
- A weekly asynchronous update on the issue board instead of long meetings.

9. Measurable Success Criteria

Targets are illustrative (Assumption A7). Real baselines should be measured and targets adjusted.

#	Metric	How measured	Target
S1	Direct pushes to main	GitLab audit events	0
S2	Merged MRs with required approval	MR analytics	100%
S3	Median time to first review	MR analytics / API	8 hours or less
S4	Median MR cycle time (open to merge)	MR analytics / Value Stream Analytics	2 days or less
S5	MRs linked to an issue	MR and issue links	95% or more
S6	Pipeline pass rate on merged MRs	CI/CD analytics	100%
S7	Releases with a version tag and notes	Releases page	100%
S8	Secrets found in repositories	Secret detection reports	0 open
S9	Critical bugs in production, 30 days after release	Issue tracker	2 or fewer
S10	Developers who rate the workflow as helpful	Short survey	80% or more

10. Recommendations and Lessons Learned

10.1 Recommendations

1. Start every task as an issue with clear acceptance criteria.
2. Use short-lived branches named after the issue and keep merge requests small.
3. Never allow direct pushes to main or develop; enforce this with protected branches.
4. Use CODEOWNERS and a review time target so reviews flow across time zones.
5. Let automation do the checking: build, tests, linting and security scans on every MR.
6. Tag every release with a semantic version and publish release notes.
7. Review the workflow every quarter and adjust rules as the team grows.

10.2 Lessons Learned

These come from designing this solution, not from a live project.

- **Process works best when it is enforced by tools**, not by reminders.
- **Remote teams need written clarity**: good issue and MR templates replace many meetings.
- **Small changes are safer**: small MRs are reviewed faster and break less.
- **Traceability is a gift**: issue to MR to release makes every change explainable.

